Cheatsheet · 10 chapters on one page

MLOps cheatsheet

Every formula, key idea, table, memory aid and common mistake from the chapters, gathered in one place for quick revision. Nothing here is new. Each card links back to the lesson that teaches it, so open the lesson when a line does not make sense yet.

60 key ideas | 41 formulas | 29 tables | 43 tips and traps

STEP 1 Set up and plan

1.1 Set up your computer for machine learning

Python, virtual environments, Git, VS Code, Docker and GPUs set up on macOS, Windows, Linux and ChromeOS.

KEY IDEAS

- **The toolbox** Python 3.12 or newer, terminal, .venv per project, pip or uv, Git, VS Code, later Docker and a GPU or Colab.
- **PATH** The first match in PATH runs. activate puts .venv first. python -m pip keeps pip and python together.
- **Per system** macOS and Linux: python3, .venv/bin. Windows: python or py, .venv\Scripts, execution policy. ChromeOS: Linux container, Debian steps.
- **Reproducible** pip freeze > requirements.txt, pip install -r requirements.txt. Commit requirements, ignore .venv. Never sudo pip.
- **Docker** Desktop on macOS and Windows (WSL 2), Engine on Linux, docker group, docker run hello-world.
- **GPU** NVIDIA: driver, nvidia-smi, CUDA build of PyTorch. Apple silicon: MPS. Chromebook: Colab or Kaggle.

TABLES

Table: tool · what it is · why ml needs it

Tool	What it is	Why ML needs it
Python 3.12 or newer	The language and its interpreter	NumPy, pandas, scikit-learn and PyTorch need a recent Python
Terminal	A window where you type commands	Installing, running scripts, Git, Docker
Virtual environment	A folder with its own Python and packages	Each project keeps its own versions
pip or uv	Package installers	Install libraries from PyPI, the Python package index
Git	Version control	History of code and configs, teamwork, CI
VS Code	Editor with Python and Jupyter support	Write, run and debug code and notebooks
Docker	Runs apps in containers	Ship the model with everything it needs
GPU or cloud notebook	Fast parallel hardware	Training deep networks in hours, not days

Lesson: What you need and why

Table: command · pip and venv · uv

Command	pip and venv	uv
Create an environment	`python3 -m venv .venv`	`uv venv`
Install packages	`pip install pandas`	`uv pip install pandas`
Install from a file	`pip install -r requirements.txt`	`uv pip install -r requirements.txt`
Save exact versions	`pip freeze > requirements.txt`	`uv pip freeze > requirements.txt`
Install a Python version	Use the python.org installer	`uv python install 3.14`

Lesson: Virtual environments and packages

Table: problem · system · fix

Problem	System	Fix
`python: command not found`	macOS, Linux, ChromeOS	Use `python3`, or activate the .venv
Typing python opens the Microsoft Store	Windows	Turn off the App execution aliases for python
Running scripts is disabled on this system	Windows PowerShell	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
`error: externally-managed-environment`	Debian, Ubuntu, Fedora, ChromeOS	Install inside a virtual environment, never with sudo pip
ensurepip is not available	Debian, Ubuntu, ChromeOS	`sudo apt install python3-venv`
A package installed but import fails	All	pip and python come from different places: use `python -m pip`
Linux option missing in Settings	ChromeOS	Managed device: ask the administrator or use Colab
Docker permission denied	Linux, ChromeOS	Join the docker group, then log out and back in

Lesson: GPUs, the cloud and common problems

Table: · venv and pip · uv · conda

	venv and pip	uv	conda	Docker
What it isolates	Python packages	Python packages and Python versions	Python and non Python libraries	The whole system: OS libraries, Python, packages
Comes with Python	Yes	No, one small install	No, a separate distribution	No, a separate app
Speed	Fine	Very fast	Slower	Slow to build, fast to start
Lock file	requirements.txt	uv.lock or requirements.txt	environment.yml	Dockerfile plus requirements
Best for	Learning, small projects	Everyday work, CI	Science stacks with C and CUDA libraries	Serving and deploying

Lesson: venv, uv, conda and Docker compared

TIPS AND TRAPS

REMEMBER

Run this check after each part of the setup. The output above is from the computer that built this page: Python 3.14.6 on Linux inside a virtual environment, with every package installed. Inside this web page, the same program runs in the browser's own Python, so it reports a different system and that the web packages are not installed.

Lesson: What you need and why

IN THE EXAM

Two names, one idea: macOS, Linux and ChromeOS call it `python3`, Windows calls it `python` or `py`. Inside an activated virtual environment, `python` works everywhere.

Lesson: Install Python and open a terminal

IN THE EXAM

The program shows what python -m venv creates: a bin folder (Scripts on Windows), include and lib folders, and pyvenv.cfg. The new python reports that it is inside a virtual environment, its packages go to the environment's own site-packages, and include-system-site-packages = false means it cannot see packages installed for the whole computer.

Lesson: Virtual environments and packages

REMEMBER

The check finds two problems: NumPy 2.4.6 is installed but the file asks for 2.3.5, and MLflow is missing. Running pip install -r requirements.txt fixes both, which is exactly what a CI job or a new teammate does first.

Lesson: Virtual environments and packages

REMEMBER

If hello-world prints Hello from Docker!, the install works. Permission denied on /var/run/docker.sock on Linux means you have not logged out and back in since joining the docker group.

Lesson: Docker on each system

1.2 The MLOps lifecycle and reproducible projects

The MLOps loop and maturity levels, seeds and run records, data and model versioning, and project layout.

KEY IDEAS

- **MLOps** Version, test, deploy, monitor and retrain ML. DevOps plus data and models.
- **Levels** 0 manual, 1 automated training pipeline (continuous training), 2 CI/CD for the pipeline.
- **Reproducible** Fix seeds, report spread over seeds, save run records: data fingerprint, params, metrics, versions.
- **Versions** Code in Git, data by hash (DVC), models in a registry, environment in requirements or images. Lineage.
- **Tests** Data, features, model quality and behaviour, infrastructure. Notebooks to tested functions.
- **Why it matters** Models decay as data changes. Hidden technical debt lives outside the model code.

FORMULAS

An ML system — Exam favourite

$$\text{behaviour} = \text{code} + \text{data} + \text{model} + \text{configuration}$$

All four need versions.

Lesson: What MLOps is

Continuous training — Exam favourite

$$\text{new data} \rightarrow \text{retrain} \rightarrow \text{evaluate} \rightarrow \text{register} \rightarrow \text{deploy}$$

The loop of level 1.

Lesson: What MLOps is

Automation share

$$\text{automated stages} / \text{all stages}$$

A rough maturity measure.

Lesson: What MLOps is

TABLES

Table: level · what is automated · typical sign

Level	What is automated	Typical sign
0: manual	Nothing. Notebooks, files copied by hand	A model is retrained a few times a year
1: ML pipeline automation	Data, training, evaluation and registration run as one pipeline, retraining is triggered	Continuous training (CT) on new data
2: CI/CD pipeline automation	The pipeline itself is tested, built and deployed by CI/CD	Many models updated safely every week

Lesson: What MLOps is

Table: what · versioned with · example

What	Versioned with	Example
Code and configs	Git	Commit a1b2c3d
Data	Fingerprint, DVC or lakeFS	customers.csv, hash 25e2eb7b7a87
Environment	requirements.txt, lock file, Docker image tag	ml-api:1.4.0
Model	Model registry	churn-model version 7, stage production
Experiments	Tracking server	MLflow run 3f9e, accuracy 0.981

Lesson: Versioning data and models

Table: test type · checks · example

Test type	Checks	Example
Data tests	Schema, ranges, missing values, label balance	age between 0 and 120
Feature tests	Transformations do what they claim	Scaled column has mean near 0 on training data
Model tests	Quality floor, behaviour on known cases	Accuracy at least 0.9, prediction unchanged when the name changes
Infrastructure tests	Pipeline runs end to end, model loads and serves	Training on 100 rows finishes, API returns a valid answer

Lesson: A project layout that grows into production

Table: · devops · mlops · dataops

	DevOps	MLOps	DataOps
What it ships	Software	Models and the pipelines that make them	Data products and pipelines
What changes behaviour	Code	Code, data and model	Data and transformations
Testing	Unit and integration tests	Plus data tests and model quality gates	Data quality checks
After release	Uptime and errors	Plus drift, accuracy and fairness	Freshness and quality of data
Typical tools	Git, CI/CD, Docker, Kubernetes	Plus MLflow, DVC, feature stores, registries	Airflow, dbt, data contracts

Lesson: DevOps, MLOps and DataOps compared

TIPS AND TRAPS

IN THE EXAM The first block changes every time you run it, which is the point. With the seed fixed at 42, five runs give exactly the same accuracy, 0.972, five times. Over ten different seeds the accuracy has mean 0.956 and standard deviation 0.014, ranging from 0.937 to 0.972, so a difference of 0.01 between two models trained once means nothing.

Lesson: Reproducibility: seeds and run records

REMEMBER The record says the wine data had 178 rows and 13 features with fingerprint 3a28849ef236, lists the parameters (C = 0.5, seed 7), the metrics (test accuracy 0.9778) and the Python, NumPy and scikit-learn versions. With this one JSON file, anyone can rebuild and check the model.

Lesson: Reproducibility: seeds and run records

IN THE EXAM The original breast cancer data gives a 5 fold accuracy of 0.981. Flipping 40 labels (7 percent) drops it to 0.882 with the same number of rows. Losing about 30 percent of the rows (399 left) gives 0.975. Each version has a different fingerprint, so the run record shows exactly which one a model saw.

Lesson: Versioning data and models

STEP 2 Build and ship

2.1 Pipelines and saving models

Leak free scikit-learn pipelines, saving with joblib and metadata, file formats, and input contracts.

KEY IDEAS

- **Pipeline** Preprocessing plus model in one object. ColumnTransformer: numbers impute and scale, categories one hot.
- **fit and transform** Steps learn on training rows only, reuse at predict. handle_unknown='ignore' for unseen categories.
- **Leakage** No step may see test rows. Selection outside CV: 0.85 on noise, inside: 0.48. Split by time and group too.
- **Saving** joblib.dump the whole pipeline plus JSON metadata (features, classes, versions). Load trusted files only.
- **Formats** joblib for own Python services, skops for safe sharing, ONNX for other languages.
- **Input contract** Check presence, type, range, unknown categories, extra fields. Reject with 422.

FORMULAS

Standard scaling Exam favourite

$$z = (x - \mu_{train}) / \sigma_{train}$$

Statistics from the training data only.

Lesson: One pipeline from raw columns to prediction

Median imputation Exam favourite

$$\text{missing } x \rightarrow \text{median of training } x$$

Robust to outliers.

Lesson: One pipeline from raw columns to prediction

One hot columns Exam favourite

$$\text{one column per category seen in training}$$

Unknown category → all zeros with ignore.

Lesson: One pipeline from raw columns to prediction

TABLES

Table: kind of leakage · example · fix

Kind of leakage	Example	Fix
Preprocessing on all rows	Scaler fitted before the split	Scaler inside the pipeline
Selection on all rows	Top correlated columns chosen with test rows	SelectKBest inside the pipeline
Target leakage	A column filled in after the outcome is known	Remove it, check the time each column becomes available
Time leakage	Random split of time ordered data	Split by time: train on the past, test on the future
Group leakage	The same patient in train and test	GroupKFold by patient

Lesson: Leakage: why every learning step belongs inside the pipeline

Table: · joblib (pickle) · skops · onnx

	joblib (pickle)	skops	ONNX	Framework format
What it stores	Any Python object	scikit-learn objects	A computation graph	PyTorch state_dict, Keras .keras
Safe to load from strangers	No, can run code	Yes, checks types first	Yes	Mostly (PyTorch weights_only loading)
Needs Python to serve	Yes, same versions	Yes	No: C++, C#, Java, JavaScript runtimes	Usually
Speed of serving	Normal	Normal	Often faster with ONNX Runtime	Normal
Best for	Your own models, quick start	Sharing scikit-learn models	Serving across languages and devices	Deep learning models

Lesson: Ways to save a model compared

TIPS AND TRAPS

IN THE EXAM

The churn pipeline reaches a 5 fold ROC AUC of 0.751 and a test accuracy of 0.69. After preprocessing, 5 raw columns have become 9 (3 scaled numbers and 6 one hot columns, with names such as cat__plan_basic). A new customer with a missing bill and a plan called student, which the model never saw, still gets a churn probability of 0.782 instead of an error.

Lesson: One pipeline from raw columns to prediction

IN THE EXAM

On 100 rows of random noise with random labels, choosing the 20 best of 5000 columns before cross validation reports an accuracy of 0.85. Doing the same selection inside the pipeline reports 0.48, the honest answer for labels that are pure chance.

Lesson: Leakage: why every learning step belongs inside the pipeline

IN THE EXAM

The fitted iris pipeline takes about 1.5 KB on disk. After loading, its predictions match the original on every row, and the flower [5.9, 3.0, 5.1, 1.8] is predicted as virginica. The JSON file records scikit-learn 1.8.0, which the serving side should match.

Lesson: Saving and loading a trained pipeline

REMEMBER

The first request passes. The second is rejected because tenure is −3 and the bill was sent as text. The third has four problems: a missing bill, 2.5 support calls where a whole number is required, an unknown plan, and an unexpected coupon field.

Lesson: An input contract for the model

2.2 Serving models with Flask, FastAPI and Django

Model APIs in Flask, FastAPI and Django, calling them on each OS, online against batch, and production servers sized with Little's law.

KEY IDEAS

- **Model API** Load once. /health plus POST /predict. JSON in and out. 200 success, 422 bad content, 405 wrong method, 500 is a bug.
- **Frameworks** Flask small. FastAPI checks Pydantic classes, docs at /docs. Django for a model inside a full web app.
- **Test then call** Test client in one process. Then curl (macOS, Linux, ChromeOS) or Invoke-RestMethod and curl.exe (Windows).
- **Online or batch** Online answers during the visit. Batch scores on a schedule, cheaper, less fresh. Read big files in chunks.
- **Production servers** Gunicorn on macOS and Linux, Waitress on Windows, Uvicorn for FastAPI everywhere. Several workers. Never app.run in production.
- **Capacity** L = λW. Workers = λ × latency / target use. M/M/1: W = 1/(μ − λ) explodes near capacity.

FORMULAS

Latency Exam favourite

$$\text{time from request sent to response received}$$

Report the median and the 95th or 99th percentile, not only the average.

Lesson: A model behind a web API

Throughput of one worker Exam favourite

$$\text{requests per second} = 1000 / \text{latency in ms}$$

A worker that needs 20 ms serves at most 50 requests per second.

Lesson: A model behind a web API

Little's law Exam favourite

$$L = \lambda \times W$$

L requests in progress, λ arrivals per second, W seconds each request spends in the system.

Lesson: From the development server to production

Workers needed Exam favourite

$$\text{workers} = \lambda \times \text{latency} / \text{target use}$$

Aim for about 70 to 80 percent use so short bursts do not build a queue.

Lesson: From the development server to production

Waiting time for one worker (M/M/1 queue)

$$W = 1 / (\mu - \lambda)$$

μ requests per second the worker can serve. W shoots up as λ nears μ.

Lesson: From the development server to production

TABLES

Table: status · meaning · when a model api sends it

Status	Meaning	When a model API sends it
200 OK	Success	The prediction is in the body
400 Bad Request	The request itself is broken	The body is not valid JSON at all
405 Method Not Allowed	Wrong method for this route	GET sent to a POST only route
422 Unprocessable Content	Valid JSON but the content fails the checks	A word instead of a number, a missing field
500 Internal Server Error	A bug or crash inside the service	Never on purpose. Log it and fix it.

Lesson: A model behind a web API

Table: · flask · fastapi · django

	Flask	FastAPI	Django	Model servers (BentoML, TorchServe, KServe)
Size	Small and simple	Small, typed	Large, everything included	A platform
Input checks	Write them yourself	Automatic from Pydantic classes	Forms or Django REST framework serializers	Built in per model
API docs	Add a package	Automatic at /docs	Add a package	Usually automatic
Production server	Gunicorn, Waitress	Uvicorn	Gunicorn, Waitress	Included
Extras	Many plugins	Async, dependency injection	Database, admin, users	Batching requests, GPUs, scaling, model versions
Best for	Learning, small services	New model APIs	A model inside a larger web app	Many models or heavy deep learning traffic

Lesson: Flask, FastAPI, Django and model servers compared

TIPS AND TRAPS

REMEMBER

The test client sends real requests to the app inside one Python process, so no port or network is needed. The good request gets 200 with virginica at 0.703. The broken one gets 422 and an error that tells the caller what to send. The model never sees bad data.

Lesson: A model behind a web API

IN THE EXAM

One bad request produced three precise errors at once: sepal_length must be greater than 0, sepal_width is not a number, and petal_width is missing. The batch route scored two flowers in a single predict_proba call, virginica at 0.703 and setosa at 0.981.

Lesson: FastAPI: the input contract written as a class

COMMON MISTAKE

Django gives the same answers as Flask (200 with virginica at 0.703, 422 for bad JSON), and @require_POST turns a GET into 405 without running the view. In a real project the view lives in an app inside `django-admin startproject`, and the settings come from environment variables, never a SECRET_KEY written in the code.

Lesson: Django: when the model joins a full web application

REMEMBER

20,000 rows are scored in 4 chunks of 5,000. The last line times both styles on your own device, so the numbers change from run to run, but one call per row is always many times slower because each call pays the same fixed overhead.

Lesson: Online against batch prediction

IN THE EXAM

A queue does not grow in a straight line. One worker that can serve 50 requests per second answers in 25 ms at 10 per second, 100 ms at 40 per second, and 1 full second at 49 per second. That is why services keep spare capacity.

Lesson: From the development server to production

2.3 Docker and CI/CD for models

A real Dockerfile and Compose file for a model API, per OS build steps, model behaviour tests and a GitHub Actions pipeline.

KEY IDEAS

- **Image and container** Image = OS layer, Python, packages, code, model. Container = running copy. Registry stores images. -p host:container.
- **Dockerfile order** Base, requirements, pip install, then code. Code change: 2.2 s instead of 32 s. Non root USER, HEALTHCHECK, CMD.

- **Per system** Docker Desktop on macOS and Windows (WSL 2), Engine plus docker group on Linux, Engine in the Linux environment on ChromeOS. Apple silicon: --platform linux/amd64.
- **Model tests** Quality gate, known examples, invariance, directional, latency. A constant model passes invariance, so use several kinds.
- **CI/CD** GitHub Actions: install, train, pytest, build, smoke test, push on main. First failure stops. Tag images by commit, secrets never in files.
- **Numbers** Flaky suite $(1 - p)^n$. Accuracy SE $\sqrt{(p(1 - p)/n)}$. Canary $n = 4p(1 - p)/d^2$.

FORMULAS

Chance a suite of flaky tests is green

Exam favourite

$$P(\text{green}) = (1 - p)^n$$

n independent tests that each fail at random with probability p. Ten tests at 2 percent: $0.98^{10} = 0.817$.

Lesson: Testing a model, not only the code

Uncertainty of a test accuracy

Exam favourite

$$SE = \sqrt{(p(1 - p) / n)}$$

With p = 0.9111 on n = 45 test rows, SE = 0.042, so a gate at 0.90 is less than one standard error away.

Lesson: Testing a model, not only the code

TABLES

Table: word · what it is · command

Word	What it is	Command
Image	A read only package built from a Dockerfile, made of layers	`docker build -t iris-api:1.0.0 .`
Container	A running copy of an image, with its own file system and network	`docker run -p 8000:8000 iris-api:1.0.0`
Tag	A name and version for an image	`iris-api:1.0.0`
Registry	A store for images that servers pull from	`docker push ghcr.io/owner/iris-api:1.0.0`
Port mapping	Connects a port on your computer to a port in the container	`-p 8000:8000` means computer 8000 to container 8000

Lesson: Containers, images and the Dockerfile

Table: test · question it answers · in this project

Test	Question it answers	In this project
API test	Do the routes answer correctly?	test_health, test_known_flower
Input test	Is bad input rejected?	test_bad_input_is_rejected (422)
Quality gate	Is the model good enough to ship?	test_accuracy_gate (at least 0.90)
Invariance	Does tiny noise leave the answer alone?	test_small_noise_keeps_the_class
Directional	Does a change push the answer the right way?	test_longer_petals_mean_virginica
Latency	Is it fast enough?	test_latency_budget (under 50 ms)

Lesson: Testing a model, not only the code

Table: · virtual environment · docker container · virtual machine

	Virtual environment	Docker container	Virtual machine
Pins	Python packages only	Packages, Python, system libraries and OS files	A whole operating system and kernel
Start time	Instant	About a second	Tens of seconds
Size	Hundreds of MB	Hundreds of MB, shared layers	Many GB
Same on every computer	No: OS and Python can differ	Yes, for the same processor type	Yes
Isolation	None	Process level, shares the host kernel	Strong, own kernel
Best for	Writing and testing code	Shipping services, CI, Kubernetes	Strict isolation or a different OS

Lesson: Virtual environment, container or virtual machine

TIPS AND TRAPS

IN THE EXAM Real timings of this project: with the good order, a one line change in app.py rebuilt in 2.2 seconds in total, because pip install came from the cache. With `COPY . .` before pip install, the same change took 32 seconds, because the 18 second install ran again.

Lesson: Containers, images and the Dockerfile

REMEMBER The probability here is 0.653, not the 0.703 of MLOps 2.2, because train.py holds back 30 percent of the flowers for testing and trains on the rest. The image takes 555 MB on disk but only 129 MB is downloaded, because layers are stored compressed.

Lesson: Build and run the model container on your computer

COMMON MISTAKE The broken model scored 0.2 and failed three of the four checks, so CI blocks it. Look closely: it still passed the invariance test, because a model that ignores its input is perfectly stable. One kind of test is never enough.

Lesson: Testing a model, not only the code

STEP 3 Operate and improve

3.1 Experiment tracking and the model registry

Tracking runs, MLflow with per OS steps, the winner's curse, and a model registry with promotion and rollback.

KEY IDEAS

- **Tracking** Every run: parameters, metrics, artifacts, data fingerprint, Git commit, environment. Compare only runs on the same data.
- **MLflow** start_run, log_params, log_metrics, sklearn.log_model, search_runs. mlflow ui --backend-store-uri sqlite:///mlflow.db, port 5001 on macOS.
- **Selection bias** Best of many on one validation set flatters itself (0.615 vs 0.530). Untouched test set, report once. Gap about $\sigma\sqrt{(2 \ln n)}$.
- **Registry** Frozen numbered versions with lineage, movable aliases. Services load models:/name@champion.
- **Promotion** Same held out data, margin gate above noise, behaviour and fairness tests, human approval, logged reason.
- **Rollback** Move the alias back. No retraining, no rebuild.

FORMULAS

Random search coverage

Exam favourite

$$P(\text{at least one of n runs in the top q}) = 1 - (1 - q)^n$$

60 random runs reach the top 5 percent with 95 percent chance.

Lesson: Choosing the best run without fooling yourself

Expected best of n noisy scores

Exam favourite

$$\text{about } \sigma \sqrt{(2 \ln n)} \text{ above the mean}$$

With σ = 0.02 and n = 100: 0.061, close to the 0.064 drop in the animation.

Lesson: Choosing the best run without fooling yourself

TABLES

Table: what to log · example · why

What to log	Example	Why
Parameters	C = 1.0, class_weight = None	To know what produced the result
Metrics	val_auc = 0.9952	To compare and rank runs
Artifacts	model file, confusion matrix plot	To reuse the exact model later
Data version	a hash of the training data, or a DVC tag	Scores on different data cannot be compared
Code version	Git commit id	To rerun exactly the same code
Environment	Python and package versions	The same code can behave differently with other versions

Lesson: Experiment tracking: a record of every run

Table: · mlflow · weights and biases · dvc

	MLflow	Weights and Biases	DVC	TensorBoard
Type	Open source platform	Hosted service (also self hosted)	Open source, Git based	Open source viewer
Strength	Tracking, registry and model packaging in one	Rich dashboards and team features	Versions data and pipelines next to Git	Training curves for deep learning
Model registry	Yes	Yes	Through Git tags and its studio	No
Works offline	Yes	Mostly needs its server	Yes	Yes
Best for	A free, standard stack	Teams who want polish and hosting	Data versioning and reproducible pipelines	Watching one training run

Lesson: Tracking tools compared

TIPS AND TRAPS

REMEMBER

Each line of runs.jsonl is one complete record: settings, score, a 12 character fingerprint of the training data and the Git commit. Because every run has the same data fingerprint, their scores are comparable. MLflow stores the same facts in a database instead of a text file.

Lesson: Experiment tracking: a record of every run

IN THE EXAM

Four runs were logged and sorted by validation AUC. C = 1.0 won with 0.9952, was registered as version 1 of breast-cancer-classifier with the alias champion, and loading models:/breast-cancer-classifier@champion gave back exactly the same AUC.

Lesson: MLflow on your own computer

COMMON MISTAKE

The labels are coin flips, so every model is truly 50 percent accurate. The best of 1000 runs still showed 0.615 on validation, then 0.530 on fresh test data. With 200 validation rows one standard error is 0.035, and the best of 1000 lands about three standard errors above the truth.

Lesson: Choosing the best run without fooling yourself

IN THE EXAM v2 (AUC 0.8844) was kept but not promoted. v3 (0.9942) beat v1 (0.9555) and became champion. After the incident, champion moved back to v1 in one line, with no retraining and no rebuild.

Lesson: The model registry: versions, aliases and promotion

3.2 Monitoring and drift

What to monitor, PSI and KS drift tests, concept against data drift, retraining, and a scheduled drift job on every OS.

KEY IDEAS

- **Four layers** Service (p50, p95, p99, errors), data (missing, schema, drift), model (scores, accuracy), business. Percentiles, not means.
- **SLO and budget** Budget = (1 − SLO) × requests. Burn rate = actual / allowed error rate.
- **Drift types** Data (inputs move), label (class shares move), concept (rule changes, invisible to input checks).
- **PSI and KS** PSI = Σ(q − p) ln(q/p): 0.1 watch, 0.25 drift. KS = max CDF gap, very sensitive to sample size.
- **Decay** Concept drift: accuracy 0.95 to 0.57 with PSI 0.01. Data drift: PSI 6.2, accuracy 0.95. Retrain on schedule or trigger through the gate.
- **Scheduling** cron or launchd (macOS, Full Disk Access for Documents), schtasks (Windows), cron or systemd timer (Linux), GitHub Actions schedule anywhere.

FORMULAS

Percentile Exam favourite

p99 = the value that 99 percent of requests stay below

Tail latency is what users notice.

Lesson: What to watch once a model is live

Error budget Exam favourite

allowed failures = (1 − SLO) × requests

99.9 percent of 100,000 leaves a budget of 100 errors.

Lesson: What to watch once a model is live

Burn rate

actual error rate / allowed error rate

A burn rate of 5 uses a 30 day budget in 6 days.

Lesson: What to watch once a model is live

Population stability index Exam favourite

$$PSI = \Sigma_{bins} (q_i - p_i) \ln(q_i / p_i)$$

p training shares, q live shares. Below 0.1 stable, 0.1 to 0.25 watch, above 0.25 drift.

Lesson: Drift: when live data stops looking like training data

Kolmogorov Smirnov statistic Exam favourite

$$D = \max_x |F_{train}(x) - F_{live}(x)|$$

Largest gap between the two cumulative curves.

Lesson: Drift: when live data stops looking like training data

PSI and KL divergence

$$PSI = KL(q \,\|\, p) + KL(p \,\|\, q)$$

For two normals with the same spread: $(\Delta\mu / \sigma)^2$.

Lesson: Drift: when live data stops looking like training data

TABLES

Table: layer · examples · needs true labels?

Layer	Examples	Needs true labels?
Service	p50, p95, p99 latency, error rate, requests per second	No
Data	missing share, unseen categories, out of range values, PSI per feature	No
Model	prediction mean and spread, PSI of the scores, accuracy, calibration	Accuracy needs labels
Business	fraud losses, click through rate, churn	Usually, and with delay

Lesson: What to watch once a model is live

Table: retraining strategy · when it fits · cost

Retraining strategy	When it fits	Cost
On a schedule (weekly, monthly)	Steady, slow drift and labels arrive regularly	Simple, may retrain without need
On a trigger (accuracy or drift past a threshold)	Irregular changes	Needs reliable monitoring
Online or continual learning	Fast changing data such as ads or news	Complex, risk of learning from bad data

Lesson: Concept drift, label delay and retraining

Table: · psi · ks test · chi square

	PSI	KS test	Chi square	Wasserstein distance
Data type	Numeric (binned) or categories	Numeric	Categories	Numeric
Answers	How big is the change	Is there any change	Is there any change	How far did the values move
Sensitive to sample size	Little	Very: big samples flag tiny changes	Very	Little
Rule of thumb	0.1 watch, 0.25 drift	p below 0.05, used with care	p below 0.05, used with care	Compare with the feature's spread

Lesson: When an alert fires

TIPS AND TRAPS

COMMON MISTAKE The average latency is 24.5 ms, which sounds fine, but the 99th percentile is 292 ms, so 1 request in 100 is ten times slower than typical. Only 98.27 percent of requests met the 200 ms target, below the 99 percent SLO, while errors used 65 percent of the day's budget.

Lesson: What to watch once a model is live

IN THE EXAM A shift of 8 in the mean gives PSI 0.610, clear drift. A wider spread gives 0.246 with no change in the mean, which a check on averages would miss. With 200,000 rows a harmless shift of 0.3 already has a KS p value of 1.3e−12, while PSI is 0.001. Use the size of the change to decide, and the p value only as a hint.

Lesson: Drift: when live data stops looking like training data

IN THE EXAM Under concept drift accuracy fell from 0.95 to 0.57 in a year while the PSI of the input stayed at 0.01. Retraining every 3 months kept it between 0.86 and 0.94. Under data drift the PSI reached 6.20 but accuracy stayed at 0.94 to 0.95, because the model's rule still held.

Lesson: Concept drift, label delay and retraining

REMEMBER On the demo data the income column moved (PSI 0.472) and 6 percent of ages went missing, above the 5 percent limit, so both are flagged. The score column needs watching (0.155). With --strict the job exits with code 1, which schedulers and CI treat as a failure.

Lesson: Run a drift check every day on your computer

3.3 Explainability and fairness

Permutation importance, Shapley values and SHAP, fairness metrics by group, mitigation, impossibility results and model cards.

KEY IDEAS

- **Global importance** Permutation on test data: I = s − mean shuffled score. Impurity importance is computed on training data and favours many valued features.

- **Shapley values** φ averages what a feature adds over every order. Base + Σφ = prediction. $2^{|F|}$ coalitions, so use TreeExplainer or sampling.
- **Explanations** Show how the model used a feature, not a cause. They depend on the background data. Counterfactuals give actionable changes.
- **Group metrics** Disparate impact = rate B / rate A, flag below 0.8. Equal opportunity: equal TPR. Equalized odds: equal TPR and FPR.
- **Mitigation** Pre processing (data), in processing (constrained loss), post processing (group thresholds). Re-measure everything afterwards.
- **Impossibility and documentation** Different base rates: calibration and equal error rates conflict. Pick a rule for the harm, write a model card, gate promotion on it.

FORMULAS

Permutation importance

Exam favourite

$$I_j = s - (1/K)\,\Sigma_k\, s_{k,j}$$

s is the test score, $s_{k,j}$ the score after the k-th shuffle of feature j.

Lesson: Which features does the model use?

Impurity importance

$$I_j = \Sigma_{\text{splits on } j}\,(n_{\text{node}}/n)\,\Delta\text{impurity}$$

Measured on training data, biased towards features with many values.

Lesson: Which features does the model use?

Shapley value

Exam favourite

$$\varphi_j = \Sigma_{S \subseteq F \setminus j}\,[\,|S|!\,(|F| - |S| - 1)!\,/\,|F|!\,]\cdot[\,v(S \cup j) - v(S)\,]$$

$v(S)$ is the average prediction when only the features in S are known.

Lesson: Shapley values: why this one prediction?

Efficiency

Exam favourite

$$f(x) = \varphi_0 + \Sigma_j\,\varphi_j$$

φ_0 is the base value, the average prediction.

Lesson: Shapley values: why this one prediction?

Cost of the exact sum

$$2^{|F|}\ \text{coalitions}$$

3 features need 8, 20 features already need 1,048,576.

Lesson: Shapley values: why this one prediction?

Disparate impact ratio

Exam favourite

$$DI = P(\hat{y} = 1 \mid B)\,/\,P(\hat{y} = 1 \mid A)$$

The four fifths rule flags DI below 0.8.

Lesson: Is the model fair to every group?

Equal opportunity gap

Exam favourite

$$TPR_A - TPR_B, \text{ with } TPR = P(\hat{y} = 1 \mid y = 1)$$

Zero means qualified people are treated alike.

Lesson: Is the model fair to every group?

Equalized odds

$$TPR_A = TPR_B \text{ and } FPR_A = FPR_B$$

Both kinds of error are shared equally.

Lesson: Is the model fair to every group?

Chouldechova's identity

Exam favourite

$$FPR = [p\,/\,(1-p)]\cdot[(1 - PPV)\,/\,PPV]\cdot(1 - FNR)$$

p is the base rate. With equal PPV and FNR, different p forces different FPR.

Lesson: Fixing unfairness, and why you cannot have every kind

TABLES

Table: method · data it uses · strength · weakness

Method	Data it uses	Strength	Weakness
Impurity (tree models)	Training	Free, instant	Favours many valued features, says nothing about test data
Permutation	Test	Any model, measures real dependence	Correlated features share or hide credit
Mean |SHAP|	Any	Consistent with local explanations	Slower for non tree models

Lesson: Which features does the model use?

Table: · demographic parity · equal opportunity · equalized odds

	Demographic parity	Equal opportunity	Equalized odds	Calibration by group
Asks for equal	Approval rate	TPR	TPR and FPR	Meaning of a score
Uses true labels	No	Yes	Yes	Yes
Fits when	Labels themselves are biased	Missing a qualified person is the main harm	Both errors harm people	Scores are read as probabilities
Main cost	Can approve unqualified people	FPR may differ	Usually lowers accuracy most	Rates may differ between groups

Lesson: Fixing unfairness, and why you cannot have every kind

Table: model card section · example for the loan model

Model card section	Example for the loan model
Intended use	Rank loan applications for human review. Not for automatic refusal.
Training data	20,000 past applications, 2019 to 2024, one country
Metrics	Accuracy 0.637 overall, reported per group
Fairness	Equal opportunity gap 0.00 after group thresholds, disparate impact 0.92
Explanations	SHAP top three reasons sent with every refusal
Limits	Not tested on self employed applicants. Retrain if PSI of income passes 0.25 (MLOps 3.2).

Lesson: Model cards and responsible release

TIPS AND TRAPS

IN THE EXAM

Impurity importance always adds up to 1 and gives worst radius 0.442. On test data, shuffling worst radius costs 0.170 accuracy, the real dependence. The random ID column, pure noise, still got 0.018 of the impurity credit because its many distinct values offer many split points on training data. Shuffling it changed the test score by −0.008, which is just noise.

Lesson: Which features does the model use?

IN THE EXAM

The base value is 0.450, the average predicted chance of repaying. This applicant's debt ratio of 0.82 pulls the prediction down by 0.194, the low income by 0.121 and the young age by 0.052, ending at 0.083. The three contributions add up exactly from 0.450 to 0.083, which is the efficiency property.

Lesson: Shapley values: why this one prediction?

IN THE EXAM

With one threshold of 0.5 group B is approved 33 percent of the time against 64 percent for group A, a disparate impact of 0.52, far below the four fifths rule of 0.8. Among people who did repay, B was approved 48 percent of the time and A 75 percent. Lowering B's threshold to 0.38 closed the TPR gap, raised the ratio to 0.92 and cost only 0.010 accuracy (0.647 to 0.637).

Lesson: Is the model fair to every group?

REMEMBER

TreeExplainer works in log odds for this model, and for the first applicant 0.027 + 0.489 + 0.027 + 0.401 = 0.943, exactly the model's own log odds. Postcode gets a small mean |SHAP| of 0.055, yet the audit still shows a TPR gap of 0.196 between groups, because income also differs between groups. ThresholdOptimizer cut the gap to 0.018 while overall accuracy went from 0.672 to 0.665.

Lesson: SHAP and Fairlearn on your own computer

REMEMBER

Explainability and fairness checks belong in the same CI pipeline as accuracy tests (MLOps 2.3). A gate such as "fail if disparate impact is below 0.8 or the TPR gap is above 0.05" stops an unfair model from being promoted, in the same way as a failing unit test.

Lesson: Model cards and responsible release

STEP 4 Design and certify

4.1 Designing an ML system end to end

Sizing with QPS and Little's law, serving modes, skew, shadow and canary releases, A/B test sample size and peeking, and a local load test.

KEY IDEAS

- **Design framework** Goal and metric, requirements as numbers, data and features, baseline model, serving, offline then online evaluation, monitoring and retraining.
- **Sizing** QPS = daily requests / 86,400, times a peak factor. Little's law: throughput = workers / latency. Replicas = ceil(peak / (capacity × load)).
- **Serving modes** Batch (scheduled, stale), online (ms, tail latency), streaming (fresh features). Retrieve then rank for big catalogues.
- **Skew** Same feature, different code in training and serving. One definition or feature store, logged features, parity test.
- **Safe release** Shadow, canary 1 to 100 percent with guardrails, automatic rollback, keep the old version ready.
- **A/B tests** $n = (z_a + z_b)^2\, 2\bar{p}(1-\bar{p}) / \delta^2$. Fix n first, test once. Peeking: 4.4 to 22 percent false alarms.

FORMULAS

Average load Exam favourite

$$\text{QPS} = \text{daily requests} / 86{,}400$$

A day has 86,400 seconds.

Lesson: Start with the problem, then size the system

Replicas

$$\text{replicas} = \lceil \text{peak QPS} / (\text{per replica QPS} \times \text{target load}) \rceil$$

A target load of 0.6 leaves room for spikes and a failed replica.

Lesson: Start with the problem, then size the system

Sample size per group Exam favourite

$$n = (z_{1-\alpha/2} + z_{1-\beta})^2 \cdot 2\,\bar{p}(1-\bar{p}) / \delta^2$$

$\bar{p}$ is the average rate, δ the lift. Halving δ needs four times the users.

Lesson: A/B testing a new model

Two proportion z test Exam favourite

$$z = (p_B - p_A) / \sqrt{(\,p(1-p)(1/n_A + 1/n_B)\,)}$$

p is the pooled rate of both groups.

Lesson: A/B testing a new model

Many looks

$$P(\text{at least one false alarm}) \le 1 - (1 - \alpha)^k$$

An upper limit for k looks. Real looks share data, so the true rate is lower but still far above α.

Lesson: A/B testing a new model

TABLES

Table: · batch · online (on request) · streaming

	Batch	Online (on request)	Streaming
When it predicts	On a schedule, for everyone	When a request arrives	When an event arrives
Latency	Hours, results read from a table	Milliseconds	Seconds
Feature freshness	As old as the last run	Request data plus stored features	Features updated from the event stream
Example	Nightly churn scores for the sales team	Fraud check during a payment	Live delivery time estimates
Main risk	Stale scores, wasted work for inactive users	Tail latency and cost	Complex pipelines (Kafka, Flink)

Lesson: Batch, online and streaming predictions

Table: strategy · users see the new model · good for · watch out for

Strategy	Users see the new model	Good for	Watch out for
Shadow	No	Latency, crashes, agreement with the old model	Cannot measure user reactions, doubles compute
Canary	A small growing share	Catching bugs with little harm	Small shares need time to show rare problems
Blue-green	All at once, with an instant switch back	Simple, fast rollback	No gradual exposure
A/B test	A fixed random half (or share)	Measuring the effect on a business metric	Needs enough users and a fixed plan
Multi-armed bandit	More traffic to the version that is winning	Many variants, short campaigns	Harder to read the final effect

Lesson: Shadow, canary and rollback

Table: part of the design · choice · why · chapter

Part of the design	Choice	Why	Chapter
Metric	Fraud losses saved, with a cap on wrongly blocked good payments	Accuracy is useless at 0.1 percent fraud	ML 2.x metrics
Features	Payment amount, merchant, card history counts over 1 hour and 30 days	History counts come from a feature store, updated from the payment stream	This chapter
Model	Gradient boosted trees, a few milliseconds per payment	Strong on tabular data, fast, explainable with SHAP	MLOps 3.3
Serving	Online, inside the payment flow, rules as a fallback	Must decide before the payment completes	MLOps 2.2
Rollout	Shadow for two weeks, then canary with guardrails on block rate	Fraud labels are slow, so start with safe signals	This chapter
Monitoring	Score PSI, block rate, latency p99, dispute rate when labels arrive	Fraudsters adapt, so concept drift is certain	MLOps 3.2
Retraining	Weekly through the tested pipeline and the registry gate	New fraud patterns appear all the time	MLOps 3.1

Lesson: Worked design: real time fraud detection

TIPS AND TRAPS

IN THE EXAM

Ten million daily users with 20 requests each average 2,315 requests per second and peak at 6,944. One replica with 4 workers at 40 ms handles 100 per second, so 116 replicas keep each at 60 percent load. Ranking a million items per request would take a full second, which is why recommenders retrieve 500 candidates first.

Lesson: Start with the problem, then size the system

COMMON MISTAKE

The model scored 0.730 offline. The serving code filled missing spend with 0 instead of the training median and received spend in pence, not pounds, so live accuracy fell to 0.550 and the mean predicted churn collapsed from 0.435 to 0.124. A parity test on the same raw rows finds the difference on 100 percent of rows before release.

Lesson: Batch, online and streaming predictions

IN THE EXAM

To detect a lift from 5.0 to 5.5 percent with $\alpha = 0.05$ and 80 percent power you need 31,235 users per group. In 2,000 A/A tests a single look at the end gave 4.4 percent false alarms, close to the promised 5. Stopping on the first day with $p < 0.05$ gave 22.0 percent, more than four times as many.

Lesson: A/B testing a new model

IN THE EXAM

The built-in demo server can work on 4 requests at once, each taking 20 ms, so Little's law predicts 4 / 0.020 = 200 requests per second. The test gets close to 200 with 4 users and stays just under 200 with 8 and 16 users, while the median latency rises from about 21 ms to about 40 ms and then higher. Past the saturation point, more users only add waiting time. Timings change a little on every run.

Lesson: Load test your model service on your own computer

REMEMBER

The real FastAPI container from MLOps 2.3 levelled off at about 1,180 requests per second from 8 users, while its p50 went from 2.1 ms to 12.1 ms at 16 users. For the 6,944 requests per second peak, that machine would need about 10 such replicas at 60 percent load, a number you can now defend.

Lesson: Load test your model service on your own computer

REMEMBER Interviewers look for the trade-offs, not one right answer. Say what you would measure, what you would try first, and how you would know it failed.

Lesson: Worked design: real time fraud detection

4.2 MLOps revision and certification practice

The whole subject as one loop, maturity levels, a release gate across all chapters, a capstone, cloud service mapping and exam practice.

KEY IDEAS

- **The loop** Setup, lifecycle, pipelines, serving, Docker and CI/CD, tracking, monitoring, explain and fair, system design, then retrain.
- **Maturity** Level 0 manual, level 1 continuous training through a pipeline, level 2 CI/CD for the pipeline itself.
- **Release gate** Parity, quality gain on the same holdout, fairness (DI ≥ 0.8, TPR gap), score PSI, size. Any FAIL blocks promotion.
- **Key numbers** PSI 0.1 and 0.25. DI 0.8. Burn rate 14.4 for 2 percent in an hour. Peeking 4.4 to 22 percent. Layer cache 2.2 s against 32 s.
- **Capstone** Fingerprint, pipeline, tracked run, alias, tested API (200, 422), drift check, model card.
- **Cloud mapping** Pipelines, endpoints, tracking, registry, monitoring, explainability and feature store exist on Vertex AI, SageMaker and Azure ML under their own names.

FORMULAS

PSI Exam favourite

$$PSI = \Sigma\,(q - p)\,\ln(q / p)$$

0.1 watch, 0.25 drift (3.2).

Lesson: One release gate that uses every chapter

Little's law Exam favourite

$$\text{workers} = \text{request rate} \times \text{latency}$$

Sizing workers and replicas (2.2, 4.1).

Lesson: One release gate that uses every chapter

Disparate impact Exam favourite

$$DI = \text{rate}_B / \text{rate}_A \geq 0.8$$

Four fifths rule (3.3).

Lesson: One release gate that uses every chapter

Error budget and burn rate Exam favourite

$$\text{budget} = (1 - \text{SLO}) \times \text{requests}, \text{burn} = \text{actual} / \text{allowed}$$

A burn rate of 14.4 spends 2 percent of a 30 day budget in an hour (3.2).

Lesson: One release gate that uses every chapter

Shapley efficiency

$$f(x) = \varphi_0 + \Sigma\,\varphi_j$$

Contributions add up to the prediction (3.3).

Lesson: One release gate that uses every chapter

A/B sample size

$$n = (z_a + z_b)^2\, 2\bar{p}(1 - \bar{p}) / \delta^2$$

Fix n before the test (4.1).

Lesson: One release gate that uses every chapter

TABLES

Table: chapter · key idea · classic trap

Chapter	Key idea	Classic trap
1.1 Setup	A venv per project, exact versions in requirements.txt	Installing into the system Python, committing .venv
1.2 Lifecycle	Seeds, run records, data fingerprints	Reporting the best seed instead of the spread
2.1 Pipelines	Fit on training rows only, save one pipeline	Scaling or selecting features before the split
2.2 Serving	Validate input, 422 for bad requests	Loading the model inside the request handler
2.3 Docker, CI/CD	Requirements before code, gate every release	Running as root, testing only the code and not the model
3.1 Tracking	Log every run, aliases for promotion	Choosing the best of many runs on the validation set and reporting that score
3.2 Monitoring	PSI for size, labels for accuracy	Trusting KS p values on huge samples, missing concept drift
3.3 Explain, fair	Permutation importance on test data, rates by group	Thinking that dropping the group column removes bias
4.1 System design	Numbers first, safe rollout, honest A/B test	Stopping a test the first day $p < 0.05$

Lesson: The whole subject in one loop

Table: level · what is automated · what starts a new model · tools from this subject

Level	What is automated	What starts a new model	Tools from this subject
0 Manual	Only serving	A person, often months apart	Notebooks, a saved model file (2.1)
1 Pipeline automation	Data checks, training, validation, delivery	Schedule, new data or a drift alert	Pipelines (2.1), tracking and registry (3.1), monitoring (3.2)
2 CI/CD automation	Also testing, building and deploying the pipeline	Also every code change	Docker and GitHub Actions (2.3), registry gates (3.1)

Lesson: MLOps maturity levels

Table: idea (chapter) · open source here · google cloud · aws

Idea (chapter)	Open source here	Google Cloud	AWS	Azure
Pipelines (2.1, 2.3)	scikit-learn Pipeline, GitHub Actions	Vertex AI Pipelines	SageMaker Pipelines	Azure ML pipelines
Serving (2.2)	FastAPI, Docker	Vertex AI endpoints	SageMaker endpoints	Azure ML online endpoints
Tracking (3.1)	MLflow	Vertex AI Experiments	SageMaker with managed MLflow	Azure ML with MLflow
Registry (3.1)	MLflow Model Registry	Vertex AI Model Registry	SageMaker Model Registry	Azure ML model registry
Monitoring (3.2)	PSI job, cron	Vertex AI Model Monitoring	SageMaker Model Monitor	Azure ML model monitoring
Explain and fairness (3.3)	SHAP, Fairlearn	Vertex Explainable AI	SageMaker Clarify	Responsible AI dashboard
Feature store (4.1)	one shared feature function	Vertex AI Feature Store	SageMaker Feature Store	Azure ML managed feature store

Lesson: The same ideas on cloud platforms

TIPS AND TRAPS

REMEMBER

Level 2 is not always the goal. A small team with one model retrained monthly may be well served by level 1. Match the automation to how often the data and the code change.

Lesson: MLOps maturity levels

IN THE EXAM

Challenger 1 has the higher AUC, 0.0498 above the champion, because it uses postcode and the past labels favoured group A. The gate blocks it: the equal opportunity gap is 0.227 and the disparate impact is 0.48. Challenger 2 drops postcode, still gains 0.0125 AUC, passes all six checks and is promoted. A higher score on biased labels is not a better model.

Lesson: One release gate that uses every chapter

REMEMBER

The test AUC is 0.7038 on synthetic data. The API returns 200 for a good request, 422 for a negative tenure and still 200 for the unseen plan family, because the encoder ignores unknown categories. After a 20 percent price rise the PSI of monthly_bill is 0.334, so the drift check says drift. The model card records all of it.

Lesson: Capstone: the whole lifecycle on your computer

IN THE EXAM

A common exam pattern: a scenario with stale models and manual retraining, and four services to choose from. The answer is the pipeline plus monitoring service of that platform, because the problem is level 0 maturity, not the model.

Lesson: The same ideas on cloud platforms